Backend Detailed Design — Project Structure & API Behavior

Companion to Backend Spec v2.0 · Development reference

This document gives the concrete project structure for both services and the detailed behavior of every API — request, response, step-by-step logic, side effects, and errors — so it can be implemented endpoint by endpoint.

Part 1 — Project structure

1.1 Java core service (`backend-core/`)

```
backend-core/
├── build.gradle                     Spring Boot 3.3, Java 21
├── Dockerfile
├── docker-compose.yml               postgres + pgvector, vault, core, ai
├── src/main/resources/
│   ├── application.yml               shared config
│   ├── application-local.yml         local profile (localhost, dev Vault)
│   ├── application-remote.yml        remote profile (HTTPS, prod Vault)
│   └── db/migration/                 Flyway: V1__init.sql, V2__seed_curriculum.sql …
└── src/main/java/com/prep/
    ├── PrepApplication.java
    ├── config/
    │   ├── SecurityConfig.java       JWT filter chain, route rules
    │   ├── VaultConfig.java          Spring Cloud Vault — load secrets at boot
    │   ├── CorsConfig.java           allow UI origin
    │   ├── JacksonConfig.java        JSON, java.time
    │   ├── SchedulingConfig.java     @EnableScheduling
    │   ├── OpenApiConfig.java        swagger contract for the UI
    │   └── AiClientConfig.java       WebClient bean → Python service (service token)
    ├── common/
    │   ├── error/                    ApiException, ErrorCode, GlobalExceptionHandler, ErrorResponse
    │   ├── security/                 JwtService, JwtAuthFilter, @CurrentUser resolver
    │   └── dto/                      MetaResponse, PageResponse
    ├── user/                         AppUser, UserSettings (entities), repos,
    │                                 AuthController, MeController, AuthService, UserService, dto/
    ├── onboarding/                   ResumeProfile entity + repo, OnboardingController,
    │                                 OnboardingService, dto/
    ├── study/                        Phase, Week, Topic entities + repos,
    │                                 StudyController, TopicController, ProgressController,
    │                                 StudyService, ProgressService, ReadinessCalculator, dto/
    ├── content/                      Note, Attachment, Resource, Exercise, Question + repos,
    │                                 Note/Resource/Exercise/Question controllers,
    │                                 NoteService, ContentService, dto/
    ├── storage/                      FileStore (iface), LocalFileStore, S3FileStore
    ├── interview/                    Interview, InterviewQuestion, ReviewFlag, PrepPack + repos,
    │                                 InterviewController, PrepPackController,
    │                                 InterviewService, FeedbackLoopService, PrepPackService, dto/
    ├── star/                         StarStory + repo, StarController, StarService
    ├── mock/                         MockSession + repo, MockController, MockService
    ├── review/                       ReviewController, PlanController,
    │                                 ReviewService, SpacedRepetitionService, AdaptivePlanService
    ├── jobs/                         JobAlert + repo, JobController, JobService,
    │                                 gmail/ (GmailClient, JobAlertParser)
    ├── ai/                           AiGatewayController (/api/ai/**), AiClient (→ Python),
    │                                 BudgetGuard, UsageService, UsageLog + repo, dto/
    └── ingestion/                    scheduled: GmailPoller, JobScoringBatch,
                                      WeeklyAnalysisJob, PlanRebalanceJob
```

Layering (every module): `Controller → Service → Repository → Entity`. Controllers do no business logic (validate + delegate). DTOs cross the boundary; entities never leave the service layer. `@CurrentUser` injects the authenticated user id into every call.

1.2 Python AI service (`ai-service/`)

```
ai-service/
├── pyproject.toml                    fastapi, uvicorn, langgraph, ragas, hvac, psycopg, pgvector
├── Dockerfile
└── app/
    ├── main.py                       FastAPI app, mounts routers, service-token auth
    ├── config.py                     settings, default model ids
    ├── vault.py                      hvac client — load LLM keys, service token
    ├── db.py                         Postgres/pgvector connection (own pool)
    ├── deps.py                       verify inter-service token; load user_settings
    ├── providers/
    │   ├── base.py                   LLMProvider: complete(), embed(), tool_call()
    │   ├── anthropic_provider.py     (default)
    │   ├── bedrock_provider.py
    │   ├── openai_provider.py
    │   └── registry.py               pick provider + tier (cheap/strong) from settings
    ├── routers/
    │   ├── parse.py                  /ai/parse-resume, /ai/generate-plan, /ai/embed-resume
    │   ├── guide.py                  /ai/guide
    │   ├── score.py                  /ai/score-job
    │   ├── agents.py                 /ai/debrief, /ai/prep-pack, /ai/coach
    │   ├── mock.py                   /ai/mock/start, /ai/mock/turn
    │   └── analyze.py                /ai/analyze
    ├── rag/
    │   ├── chunker.py                resume experiences → chunks
    │   ├── embeddings.py             embed via provider
    │   └── retriever.py              pgvector similarity search
    ├── agents/
    │   ├── loop.py                   ReAct runner + guards (max steps, deadline, tokens)
    │   ├── tools.py                  tool defs; each tool = HTTP call back to Java REST
    │   ├── jobfit.py debrief.py coach.py prep_pack.py mock_agent.py
    ├── cache.py                      ai_cache read/write (check before any paid call)
    ├── usage.py                      token + cost accounting → meta block
    ├── prompts/                      parse.md, plan.md, guide.md, mock.md, …
    └── mcp/server.py                 FastMCP — same tools exposed to Claude Desktop
```

1.3 Repository / runtime layout

```
docker-compose.yml services:
  postgres   (pgvector image)  :5432
  vault      (dev or file backend) :8200
  core       (Java)            :8080   depends_on: postgres, vault
  ai         (Python)          :8000   depends_on: postgres, vault
```

Secrets seeded into Vault by a one-time `vault-seed` init container. Java reads at boot via Spring Cloud Vault; Python via `hvac`. UI talks only to `core` (8080); `ai` (8000) is internal.

Part 2 — API behavior (Java core)

Conventions: all under `/api`, JWT required except `/api/auth/**`. `@CurrentUser uid` is implicit on every authenticated call and scopes every query. Standard errors: `401` no/!valid token, `404` not found / not owned, `409` conflict, `422` validation.

2.1 Auth & me

POST `/api/auth/login`

- **Purpose:** authenticate the single user, issue tokens.
- **Request:** `{ email, password }`
- **Response:** `{ accessToken, refreshToken, expiresIn }`
- **Behavior:** load `app_user` by email → verify BCrypt hash → mint JWT (uid claim) + refresh token.
- **Errors:** `401` bad credentials.

POST `/api/auth/refresh`

- **Request:** `{ refreshToken }` → **Response:** new access token.
- **Behavior:** validate refresh token, re-issue access token. `401` if invalid/expired.

GET `/api/me`

- **Purpose:** bootstrap the app (who am I + settings + onboarding state).
- **Response:** `{ user:{email,displayName}, settings:{…}, onboarded: boolean }`
- **Behavior:** load user + `user_settings`; `onboarded = (committed plan exists)`. UI uses `onboarded=false` to route into the wizard.

2.2 Onboarding

POST `/api/onboarding/resume` (multipart)

- **Purpose:** parse an uploaded resume into a draft profile.
- **Request:** file (PDF/DOCX).
- **Response:** draft `ResumeProfile` (skills, domains, seniority, totalYears, experiences[], gaps[]).
- **Behavior:** 1) store file via `FileStore` → `file_ref`. 2) extract raw text. 3) call AI `POST /ai/parse-resume {rawText}`. 4) persist `resume_profile` with `confirmed=false`. 5) return it for the confirm step.
- **Side effects:** one AI call (logged to `usage_log`).
- **Errors:** `422` unsupported file; `502` if AI service down.

PUT `/api/onboarding/profile`

- **Purpose:** save the user's corrections to the parsed profile.
- **Request:** edited `ResumeProfile`. **Response:** saved profile.
- **Behavior:** overwrite parsed fields, set `confirmed=true`. (Parsing is lossy — this is the source of truth going forward.)

POST `/api/onboarding/plan`

- **Purpose:** generate a tailored draft plan.
- **Request:** `{ targetRole, targetLevel, weeks, hoursPerWeek, interests[], monthlyBudgetUsd, llmProvider, models }`
- **Response:** draft plan tree `{ phases:[{ weeks:[{ topics:[…] }] }] }` (not yet persisted).

- **Behavior:** 1) persist these into `user_settings`. 2) call AI `POST /ai/generate-plan {profile, targets}` — returns phases/weeks/topics including auto-leveled DSA, standard tracks (system design, behavioral), and interest add-ons (e.g. agentic-ai), each topic tagged with `source`. 3) return draft for editing. **Not committed yet.**
- **Side effects:** AI call (logged).

PUT `/api/onboarding/plan/commit`

- **Purpose:** persist the (edited) plan and finish onboarding.
- **Request:** final plan tree (user may have added/removed/reordered).
- **Behavior:** 1) insert `phase` / `week` / `topic` rows for the user. 2) trigger AI `POST /ai/embed-resume` (one-time, builds `resume_chunk` vectors for RAG). 3) mark onboarded.
- **Response:** `{ committed:true }`.

2.3 Study plan & topics

GET `/api/phases`

- **Purpose:** render the whole plan tree (sidebar + weeks + topics).
- **Response:** `[{ code,name,icon,blurb, progress:{done,total}, weeks:[{ code,title, topics:[{slug,code,title,tag,source,status,confidence}] }] }]` — **light** (no deep-dive/notes).
- **Behavior:** fetch user's phases→weeks→topics ordered by `display_order`; compute per-phase done/total.

GET `/api/topics/{slug}`

- **Purpose:** the topic workspace payload (fills every tab in one call).
- **Response:** `TopicDetail` = topic fields + `deepDive{concept,points,angle}` + `note{contentMd,attachments[]}` + `resources[]` + `exercises[]` + `questions[]`.
- **Errors:** `404` if not owned.

POST `/api/topics`

- **Purpose:** add a custom topic ("topics can be added if required").
- **Request:** `{ weekCode, title, tag, concept?, points?, angle? }`
- **Behavior:** create `topic` with `is_custom=true, source='custom', status='todo'`; generate a unique slug; if `concept` omitted, leave empty (user can later "Generate guide").
- **Response:** created `TopicDetail`. **Errors:** `404` week not found; `422` bad tag.

PATCH `/api/topics/{slug}`

- **Purpose:** edit / mark done / set confidence.
- **Request (partial):** `{ title?, tag?, status?, concept?, points?, angle?, confidence? }`
- **Behavior:** apply changes. If `status→done`: stamp `last_reviewed_at=now`, set `confidence` to a high baseline if null. Recompute progress.
- **Response:** updated `TopicDetail`.

DELETE `/api/topics/{slug}`

- **Behavior:** delete only if `is_custom=true`; else `409` (seeded topics aren't deletable — hide/skip instead). Cascade-deletes its note/resources/exercises/questions.

GET `/api/progress`

- **Purpose:** Home dashboard data.
- **Response:** `{ overallPct, streak, tracks:[{name,readiness}], flaggedCount }`

- **Behavior:** overall done/total; streak from distinct `done` action days; per-track readiness via `ReadinessCalculator` (§3.2); count open `review_flag`s.

2.4 Notes / resources / exercises / questions

GET `/api/topics/{slug}/note`

- **Response:** `{ contentMd, updatedAt, attachments:[{id,url,originalName,mimeType}] }` (`url` = `/api/attachments/{id}`). Returns empty note if none yet.

PUT `/api/topics/{slug}/note`

- **Purpose:** autosave the markdown note (Notion-style editor).
- **Request:** `{ contentMd }` → upsert (create note row if absent), set `updated_at`.

POST `/api/topics/{slug}/note/attachments` (multipart)

- **Purpose:** image/file embed in a note.
- **Behavior:** store bytes via `FileStore`, insert `attachment` row, return `{ id, url }`. UI inserts `![](attachment://{id})` (or the url) into the markdown. **Bytes never in DB.**
- **Errors:** `422` oversize/disallowed mime.

GET `/api/attachments/{id}` — stream file with its mime. `404` if not owned.

DELETE `/api/attachments/{id}` — delete row + stored file.

Resources — `GET/POST /api/topics/{slug}/resources`, `PATCH/DELETE /api/resources/{id}`

- CRUD `{ label, url, displayOrder }`. POST appends; PATCH edits; DELETE removes.

Exercises — `GET/POST /api/topics/{slug}/exercises`, `PATCH/DELETE /api/exercises/{id}`

- CRUD `{ title, repoUrl, done }`. PATCH commonly toggles `done` (optimistic in UI). `repoUrl` opens the GitHub solution; Monaco viewer reads code client-side.

Questions — `GET/POST /api/topics/{slug}/questions`, `DELETE /api/questions/{id}`

- CRUD `{ text }` — the study-side Q-bank shown on the topic screen.

2.5 Interviews & the feedback loop

GET `/api/interviews`

- **Purpose:** pipeline board.
- **Response:** `[{ id,company,role,stage,round,scheduledAt,outcome, weak:[slug] }]` grouped/sortable by stage.

GET `/api/interviews/{id}`

- **Response:** interview + `questions:[{id,text,topicSlug,selfRating}]` + `weak[]` + `jdText`.

POST `/api/interviews`

- **Request:** `{ company, role, stage, round, scheduledAt, jdText? }` (JD pasted here if the interview didn't come from a job alert). **Response:** created interview.

PATCH `/api/interviews/{id}`

- **Purpose:** move through pipeline / attach JD / set outcome.

- **Request (partial):** `{ stage?, outcome?, round?, scheduledAt?, jdText?, notes? }`.

POST `/api/interviews/{id}/questions` ★ the feedback loop

- **Purpose:** log a question that was asked and how it went.
- **Request:** `{ text, topicSlug?, selfRating(1–5) }`
- **Behavior:** 1) insert `interview_question`. 2) **if `selfRating ≤ 2`** → `FeedbackLoopService`: create `review_flag(source='interview', reason="self-rated N/5 at {company}")` for that topic and lower `topic.confidence`. 3) recompute and return `weak[]`.
- **Side effects:** flag + confidence change → the topic resurfaces in `review/today`. This is the core differentiator, guaranteed in code (the debrief agent later adds auto-tagging on top).

PATCH/DELETE `/api/interview-questions/{id}`

- Edit rating/text or remove; **re-evaluate the flag** (raise rating → resolve the flag).

2.6 Prep packs, STAR, mocks

POST `/api/interviews/{id}/prep-pack`

- **Purpose:** company-specific focus pack.
- **Behavior:** require `jdText` on the interview (else `422`). Call AI `POST /ai/prep-pack {interviewId, jd}`; the agent computes **JD topics ∩ your weak topics ∩ resume gaps**, returns `{ topics:[{topicSlug,why}], summary }`; persist `prep_pack`.
- **Response:** the pack. **GET** variant returns the stored pack.

STAR — `GET/POST/PATCH/DELETE /api/star-stories`

- CRUD `{ title, situation, task, action, result, tags[], mappedPrompts[] }`. The behavioral story bank, reused across interviews.

Mocks

- **POST** `/api/mocks/start` `{ type, scope? }` → create `mock_session` (scope defaults to current weak topics); proxies to AI `/ai/mock/start`; returns first interviewer turn.
- **POST** `/api/ai/mock/{id}` (turn) → user answer in, next question or final scoring out.
- **GET** `/api/mocks/{id}` → transcript + `score` + `feedback`.
- **Side effect:** on completion, low-scored areas create `review_flag`s (same loop as interviews).

2.7 Review (adaptive), plan, jobs, usage

GET `/api/review/today`

- **Purpose:** the daily focus strip (adaptive).
- **Response:** `[{ kind:'new'|'flagged'|'drill', topicSlug, label, note }]`
- **Behavior:** blend (a) next unstarted topics in plan order, (b) highest-priority open flags — **interview-specific flags stay scoped until that interview's date passes, then fold into the general pool**, (c) a drill from a topic that has exercises. Spaced repetition surfaces decayed-confidence topics here too.

POST `/api/review/{slug}/done`

- **Behavior:** resolve open `review_flag`s for the topic, bump `confidence`, stamp `last_reviewed_at`. Clears the loop once you've re-studied.

POST `/api/plan/rebalance`

- **Purpose:** adaptive re-plan (also runs weekly via cron).
- **Behavior:** call AI `POST /ai/coach {progress, openFlags, upcomingInterviews}`; the coach agent reorders topics and inserts reviews; apply returned changes. Plan tracks reality.

GET `/api/jobs?sort=fit|recent&status=new|interested`

- **Response:** `[{ id,company,role,location,comp,via,date,fit,reason,tags[],status }]`, default sort `fit` desc. Created by ingestion (§3.4), scored by AI; served free here.

PATCH `/api/jobs/{id}` — set `status` (interested/dismissed). Optional helper: "log as

interview" on the UI copies company/role/jdText into a new interview.

GET `/api/usage`

- **Response:** `{ monthUsd, budgetUsd, remainingUsd, warning }` for the in-app meter.
- **Behavior:** sum `usage_log.cost_usd` for the current month vs `monthly_budget_usd`.

2.8 AI gateway (Java → Python, metered)

`AiGatewayController` wraps the Python service so the UI only ever calls `/api/**`, and so budget + usage are enforced in one place.

GET `/api/ai/guide/{slug}`

- **Behavior:** 1) `BudgetGuard.tier(uid)` → strong unless over budget (then cheap + warn). 2) call AI `POST /ai/guide {topic, tier}`. 3) AI checks `ai_cache` first (free on hit). 4) write `usage_log`. 5) return `{ guideMarkdown, meta:{model,cached,budgetWarning} }`.

POST `/api/ai/mock/{slug|type}` — proxy a mock turn (same guard/usage wrapping).

POST `/api/ai/analyze` — weekly interview-pattern analysis (cached `kind=analysis`).

Part 3 — Key internal behaviors

3.1 BudgetGuard (Java)

`tier(uid)`: sum month's `usage_log.cost_usd`; if `< monthly_budget_usd` → `STRONG`, else `CHEAP` + mark `budgetWarning=true`. **Never blocks.** Every AI gateway call passes the tier to the Python service and records actual cost on return.

3.2 ReadinessCalculator (Java, deterministic)

Per track and overall: `readiness = 100 * (done/total) * (1 − 0.4*openFlags/total) * recency`, clamped 0–100, where `recency` decreases as topics' `last_reviewed_at` ages (drives spaced repetition). No AI.

3.3 FeedbackLoopService (Java)

Single place implementing the rule: a low rating (interview or mock) → `review_flag` + confidence drop; resolving a review or raising a rating clears it. Guarantees the loop in code independent of any agent.

3.4 Ingestion (Java, scheduled)

- **GmailPoller** (every 30m): query job-alert senders (read-only OAuth, token from Vault), parse postings, dedupe on `email_msg_id`, insert `job_alert(status='new')`.
- **JobScoringBatch:** collect new alerts → AI `POST /ai/score-job` (batched, cheap tier, RAG over `resume_chunk`) → write `fit_score` + `fit_reason`.
- **WeeklyAnalysisJob:** AI `/ai/analyze` over recent interviews → strategic patterns.
- **PlanRebalanceJob (weekly):** calls `/api/plan/rebalance` logic.

3.5 Python agents (ReAct, bounded)

`agents/loop.py` runs observe→think→act with guards: max steps (3–6), 30s deadline, token budget, allow-listed tools, idempotent writes, full `agent_run` trace. Tools (`agents/tools.py`) are HTTP calls back to Java REST (`search_topics`, `get_progress`, `flag_for_review`, `schedule_review`, `get_interview_log`) plus local `vector_search` (pgvector). The same tool set is exported via `mcp/server.py` for Claude Desktop — one implementation, two callers.

Part 4 — Implementation order (per endpoint group)

1. `config/` + `common/security` + `user/` + `/api/auth/**`, `/api/me` — auth works end to end.
2. `study/` + `content/` + `storage/` — `/api/phases`, `/api/topics/**`, note/resource/ exercise/question, attachments. **Wire the UI study + topic screens here.**
3. `interview/` + `review/` (FeedbackLoopService, ReadinessCalculator, `review/today`).
4. Python `ai-service` skeleton + providers + `cache` + `usage`; Java `ai/` gateway + BudgetGuard; `/api/ai/guide`.
5. `onboarding/` + AI parse/plan/embed; the wizard goes live.
6. `agents/` (jobfit→debrief→coach→prep-pack→mock) + `jobs/` + Gmail ingestion + MCP.
7. `star/`, `mock/`, `/api/usage` meter — finish remaining surfaces.